

BMEB W4020: Circuits in the Brain 2025 Fall Homework #6 Handout

Author: Shashwat Shukla, Bruce Yi Bu shashwat.shukla@columbia.edu

Based on Previous Work by: Tingkai Liu, Mehmet Kerem Turkcan, Chung-Heng Yeh, Konstantinos Psychas

Copyright 2012-2025 Shashwat Shukla, Bruce Yi Bu and Aurel A. Lazar

```
In [6]: NAME = "Andre Perez"
UNI     = "avp2141"
COLLABORATORS = "Alexander Speer - aes2376@columbia.edu"
```

PROBLEM #1:

The dendritic processing circuit in the figure below consists of a linear spatial filter with impulse response $h(x, y, t)$ in cascade with an ON-OFF Ideal Integrate-and-fire (IAF) Neuron circuit with cross and self feedbacks; the cross feedbacks are modeled as linear temporal filters with impulse response $h_{12}(t)$ and $h_{21}(t)$ and the self feedbacks are modeled with $h_{11}(t)$ and $h_{22}(t)$. The input $u(x, y, t)$ is in the *trigonometric polynomial* space.

 No description has been provided for this image

1. Write the t-transform for the filter-ON/OFF-feedback circuit in inner product form.
2. Find an algorithm for identifying the impulse response of the spatial filter, $h(x, y, t)$, using stimuli in the *trigonometric polynomial* space. Describe the process and write out any relevant equation in sufficient detail; clearly define additional notations if needed and please stay consistent (and please try to use notations consistent with the chapters).

No code is required and please use markdown for your answer.

Hint: Theorem 28 on SISO Spatial CIM in Chapter 13 is a good starting point. Also refer to Figure 13.13 in Chapter 13; the ON-OFF circuit is from Chapter 8.

Your answer here

Part 1. t-transform for the filter-ON/OFF-feedback circuit

We consider the cascade of a spatiotemporal dendritic filter $h(x, y, t)$ followed by an ON-OFF ideal IAF neuron with both self- and cross-feedback. The movie stimulus $u(x, y, t)$ is in a trigonometric polynomial space.

The filtered current

$$v(t) = \iiint h(x, y, \tau), u(x, y, t - \tau), dx, dy, d\tau$$

These are the spike trains and feedback currents

Lets make sure to have the ON and OFF spike trains be

$$\rho_1(t) = \sum_k \delta(t - t_k^1), \quad \rho_2(t) = \sum_k \delta(t - t_k^2)$$

And then we have the the temporal feedback filters producing

$$f_1(t) = (h_{11} * \rho_1)(t) + (h_{12} * \rho_2)(t)$$

$$f_2(t) = (h_{22} * \rho_2)(t) + (h_{21} * \rho_1)(t)$$

The membrane equations between the spikes

$$\kappa^1 \frac{dV_1}{dt} = b^1 + v(t) + f_1(t), \quad \kappa^2 \frac{dV_2}{dt} = -b^2 + v(t) - f_2(t)$$

ON spikes are going to occur when $V_1 = \delta^1$ and the OFF spikes when $V_2 = -\delta^2$, with having a reset to zero after each of the spikes.

Here we are integrating over the interspike intervals

ON neuron

$$\kappa^1 \delta^1 = \int_{t_k^1}^{t_{k+1}^1} (b^1 + v(t) + f_1(t)) dt$$

Rearrange so that:

$$q_k^1 = \kappa^1 \delta^1 - b^1(t_{k+1}^1 - t_k^1) - \int_{t_k^1}^{t_{k+1}^1} f_1(t) dt = \int_{t_k^1}^{t_{k+1}^1} v(t) dt$$

OFF neuron

$$\kappa^2 (-\delta^2) = \int_{t_k^2}^{t_{k+1}^2} (-b^2 + v(t) - f_2(t)) dt$$

Rearrange to get the correct t-transform for the OFF:

$$q_k^2 = -\kappa^2 \delta^2 + b^2(t_{k+1}^2 - t_k^2) + \int_{t_k^2}^{t_{k+1}^2} f_2(t) dt = \int_{t_k^2}^{t_{k+1}^2} v(t) dt$$

The inner-product form

Using the filter expression:

$$q_k^p = \iiint h(x, y, \tau) \left[\int_{t_k^p}^{t_{k+1}^p} u(x, y, t - \tau) dt \right] dx, dy, d\tau, \quad p \in 1, 2$$

Lets make sure to define the known test functions

$$\varphi_k^p(x, y, \tau) = \int_{t_k^p}^{t_{k+1}^p} u(x, y, t - \tau) dt$$

This should use the inner product

$$\langle f, g \rangle = \iiint f(x, y, \tau), g(x, y, \tau), dx, dy, d\tau$$

And then we have the t-transform becoming

$$q_k^p = \langle h, \varphi_k^p \rangle, \quad p \in 1, 2$$

Each of the ON and OFF spike intervals gives us a linear constraint on $h(x, y, t)$.

Part 2. Identification algorithm for $h(x, y, t)$ in the trig-polynomial space

1. Trigonometric polynomial basis

$$e_{l_1}(x) = \frac{1}{\sqrt{T_x}} e^{j l_1 \Omega_x x / L_1}, \quad e_{l_2}(y) = \frac{1}{\sqrt{T_y}} e^{j l_2 \Omega_y y / L_2}, \quad e_m(t) = \frac{1}{\sqrt{T_t}} e^{j m \Omega_t t / L_t}$$

The joint basis:

$$e_{l_1 l_2 m}(x, y, t) = e_{l_1}(x) e_{l_2}(y) e_m(t)$$

Expanding the filter and each of the stimulus:

$$h(x, y, t) = \sum_{l_1, l_2, m} h_{l_1 l_2 m} e_{l_1 l_2 m}(x, y, t)$$

$$u^i(x, y, t) = \sum_{l_1, l_2, m} a_{l_1 l_2 m}^i e_{l_1 l_2 m}(x, y, t)$$

2. The filtered current in the trig domain

$$v^i(t) = \sum_m \left(\sum_{l_1, l_2} a_{l_1 l_2 m}^i h_{l_1 l_2 m} \right) e_m(t) = \sum_m c_m^i e_m(t)$$

3. Make sure to apply the t-transform

$$q_{p,k}^i = \int_{t_k^{p,i}}^{t_{k+1}^{p,i}} v^i(t), dt$$

And then we should substitute the trig expansion:

$$q_{p,k}^i = \sum_m c_m^i \beta_m! \left(t_{k+1}^{p,i}, t_k^{p,i} \right)$$

The basis integrals:

$$\beta_m(t_{k+1}, t_k) = \begin{cases} \frac{\sqrt{T_t}}{j m \Omega_t / L_t} (e_m(t_{k+1}) - e_m(t_k)), & m \neq 0 \\ \frac{t_{k+1} - t_k}{\sqrt{T_t}}, & m = 0 \end{cases}$$

and let's insert c_m^i :

$$q_{p,k}^i = \sum_{l_1, l_2, m} a_{l_1 l_2 m}^i h_{l_1 l_2 m} \beta_m! \left(t_{k+1}^{p,i}, t_k^{p,i} \right)$$

Define our coefficients:

$$\phi^{i,p,k} * l_1 l_2 m = a^i * l_1 l_2 m, \beta_m! \left(t_{k+1}^{p,i}, t_k^{p,i} \right)$$

Have each spike interval yield:

$$q_{p,k}^i = \sum_{l_1, l_2, m} \phi^{i,p,k} * l_1 l_2 m, h * l_1 l_2 m$$

4. The matrix system

Vectorize the filter such that:

$$\mathbf{h} = (h_{-L_1, -L_2, -L_t}, \dots, h_{L_1, L_2, L_t})^\top$$

These are the Matrix entries:m

$$\Phi_{(i,p,k),(l_1,l_2,m)} = \phi_{l_1 l_2 m}^{i,p,k}$$

Stack all $q_{p,k}^i$ into $\mathbf{q}$.

System:

$$\mathbf{q} = \Phi, \mathbf{h}$$

5. And then finally solve for the receptive field

$$\hat{\mathbf{h}} = \Phi^\dagger \mathbf{q}$$

Then reconstruct:

$$\hat{h}(x, y, t) = \sum_{l_1, l_2, m} \hat{h} * l_1 l_2 m, e * l_1 l_2 m(x, y, t)$$

This is going to be the identified spatiotemporal receptive field $Ph(x, y, t)$.

PROBLEM #2:

You are asked to identify dendritic processing filter h in face of noise for this problem, the circuit is a simple [Filter]-[IAF] setup.



No description has been provided for this image

The dendritic processing filter $h(t)$ is *causal* linear kernel of form $1_{t \geq 0} \cdot c \cdot \frac{t}{\tau} \cdot e^{1-60 \frac{t}{\tau}}$, parameterized by $\tau = 0.2, c = 1e3$.

The threshold of the IAF, $(\delta_k), k \in \mathbb{Z}$, is a Gaussian random variable with mean $\delta = 0.01$ and variance $(0.1 \cdot \delta)^2$. Note that for each spike k the threshold is noisy. Also, choose $\kappa = 1, b = 1$.

After generating the spike sequence (t_k) above with noisy threshold above, assume that the dendritic processing filter is unknown. Identify the impulse response of the filter $\mathcal{P}h^* \approx h$ in the following steps:

1. Identify $\mathcal{P}h^*$ with stimuli of bandwidth $\Omega = 2\pi \cdot 25$ rad/s and $\Omega = 2\pi \cdot 50$ rad/s, respectively. Choose the order L such that each consecutive pair of spectrum lines are separated by a constant frequency gap (e.g. 2Hz). By having a consistent density of spectrum lines, we ensure that as we increase bandwidth of stimulus u , the frequency resolution of the identified filter $\mathcal{P}h^*$ is not compromised.
 2. Compute and plot the absolute error in dB between h , $\mathcal{P}h$ and the recovered filters $\mathcal{P}h^*$ in dB for the above 2 stimuli.
 3. Identify h with at least 5 different stimuli frequency in range $\Omega \in [2\pi \cdot 10, 2\pi \cdot 80]$ rad/s. Plot on the same figure the Mean-Squared-Error MSE ($\mathcal{P}h^*$, h) and MSE ($\mathcal{P}h^*$, $\mathcal{P}h$) for different stimulus bandwidth Ω in dB. Discuss any trends that you observed.
- Note: For this problem, it's better to compute Mean-Squared-Error Normalized by the peak of the original filter, where the error term is given as

$$err(h, Ph) = \frac{h - Ph}{\max_t |h(t)|}$$

Choose any other parameters for your experiments as appropriate.

```
In [7]: #Mount the drive and then point to HW4
from google.colab import drive
drive.mount('/content/drive')

import sys, importlib

# Make sure you're not loading a stale/local copy of compneuro
if 'compneuro' in sys.modules:
    del sys.modules['compneuro']
#the right path we should be using
sys.path.insert(0, "/content/drive/MyDrive/CCircuits/hw6")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [8]: import numpy as np
import matplotlib.pyplot as plt

from compneuro.utils.signal import spike_detect
from compneuro.neurons.ideal_integrate_fire import IdealIntegrateFire as IAF
from scipy import signal, fft
import typing as tp
from scipy import fft
np.random.seed(0)
```

```
In [9]: ### Your answer here ###
```

```
In [10]: # Basic imports
import numpy as np
import numpy.linalg as la
import matplotlib.pyplot as plt
```

```

from scipy import fft
from scipy.signal import convolve

# make sure to fix random seed so runs are repeatable
np.random.seed(0)

# Here are our helper functions

def get_coeffs_from_signal(u: np.ndarray, L: int, T: float, dt: float) -> np.ndarray:
    """Fourier coeffs for trig poly (order L, period T)."""
    coeffs_pad = fft.fftshift(fft.fft(u, n=len(u)))
    center_idx = len(u) // 2
    coeffs = dt * coeffs_pad[center_idx - L : center_idx + L + 1] / np.sqrt(T)
    return coeffs

def get_signal_from_coeffs(
    coeffs: np.ndarray,
    T: float,
    t: np.ndarray,
    normalize: bool = True,
    return_coeffs: bool = True,
):
    """Go back to time domain from trig-poly coefficients."""
    L = (len(coeffs) - 1) // 2
    coeffs_pad = np.zeros(t.shape, dtype=complex)
    coeffs_pad[:L + 1] = coeffs[L:]
    coeffs_pad[-L:] = coeffs[:L]

    u = np.real(fft.ifft(coeffs_pad, n=len(coeffs_pad))) / np.sqrt(T) * len(t)

    if normalize:
        max_amp = np.max(np.abs(u))
        u /= max_amp
        coeffs /= max_amp

    if return_coeffs:
        return coeffs, u
    return u

def random_trig_signal(
    L: int,
    T: float,
    t: np.ndarray,
    normalize: bool = True,
    rng=None,
):
    """Random trig polynomial u(t) in the space T_L."""
    rng = np.random.RandomState(rng)
    u_real = rng.randn(L)
    u_imag = rng.randn(L)
    u_dc = rng.randn()
    u_l = np.concatenate(
        [u_real + 1j * u_imag, [u_dc], np.flip(u_real - 1j * u_imag)]
    )
    u_l, u = get_signal_from_coeffs(u_l, T, t, normalize=normalize)
    return u_l, u

def project_signal(u: np.ndarray, L: int, T: float, t: np.ndarray):
    """This is the orthogonal projection of u(t) onto trig poly space (L, T)."""
    dt = t[1] - t[0]
    cff = get_coeffs_from_signal(u, L, T, dt)
    return get_signal_from_coeffs(cff, T, t, normalize=False, return_coeffs=False)

```

```

def cconv(u: np.ndarray, h: np.ndarray, dx: float = 1.0, use_fft: bool = True):
    """This is the circular convolution with step size dx."""
    Nfft = max(len(u), len(h))
    if use_fft:
        v = dx * np.real(
            np.fft.ifft(np.fft.fft(u, n=Nfft) * np.fft.fft(h, n=Nfft))
        )
        return v[: len(u)]
    v = dx * convolve(np.tile(u, 2), h, mode="full")
    return v[len(u) : len(u) + len(u)]

# this is the triangular kernel in case we don't want to plot
tri_ker = (
    lambda t, L, W, T: (2 * L + 1)
        / T
        * np.sinc((2 * L + 1) * W / (2 * L) * t / np.pi)
        / np.sinc(W / (2 * L) * t / np.pi)
)

# This is the ideal IAF encoder with random thresholds

def ideal_iaf_rt_encode(
    u: np.ndarray,
    t: np.ndarray,
    b: float,
    d: float,
    k: float,
    DeltaVar: float = 0,
    RandomInit: bool = True,
    rng: int = None,
):
    """And this is the Ideal IAF neuron with noisy threshold at each spike."""
    sigma = DeltaVar
    rng = np.random.RandomState(rng)
    dt = t[1] - t[0]

    s = np.zeros_like(t, dtype=bool) # here is our spike train (bool)
    v = np.zeros_like(t, dtype=float) # and here is our membrane voltage
    delta_v = [d + sigma * rng.randn()]

    if RandomInit:
        v[0] = 0.9 * delta_v[-1] * rng.rand()

    for tt in range(1, len(t)):
        d_V = 1.0 / k * (b + 0.5 * (u[tt] + u[tt - 1]))
        v[tt] = v[tt - 1] + dt * d_V
        if v[tt] >= delta_v[-1]:
            v[tt] -= delta_v[-1]
            s[tt] = True
            delta_v.append(d + sigma * rng.randn())

    return s, v, delta_v

# Here is the identification routine (using the finite L branch)

def identify_h_ideal_iaf_trig(
    t: np.ndarray,
    t_Ph: np.ndarray,
    u: np.ndarray,
    W: float,
    L: int,

```

```

b: float,
d: float,
k: float,
s: np.ndarray,
SmoothFactor: float = 1e-11,
RandomThreshold: bool = False,
tau=None,
rcond: float = None,
):
    """This is going to help us recover the bandlimited filter Ph* from spike times."""
    if tau is None:
        tau = [t[0], t[-1]]

    N_sig, N_t = u.shape
    dt = t[1] - t[0]

    # and these are the spike times per stimulus, restricted to [tau[0], tau[1]]
    tk = [
        t[np.logical_and.reduce([s[i], t >= tau[0], t <= tau[1]])]
        for i in range(N_sig)
    ]

    N_spk = int(np.sum(s))

    # helps us to build q vector by stacking all spike intervals
    q = np.concatenate([k * d - b * np.diff(_tk) for _tk in tk])

    # and this helps is define a finite-L setting (no Paley-Wiener side)
    T = 2 * np.pi * L / W

    def e_l(m, tvals):
        return np.exp(1j * m * W / L * tvals) / np.sqrt(T)

    u_l = fft.fft(u, axis=1) * np.sqrt(T) / len(t)

    Phi = np.zeros((len(q), 2 * L + 1), dtype=complex)

    row_idx_last = -1
    for i in range(N_sig):
        if len(tk[i]) < 2:
            continue

        # all of the rows for spikes of stimulus i
        row_idx = row_idx_last + 1 + np.arange(len(tk[i]) - 1)
        row_idx_last = row_idx[-1]

        for m in range(-L, 1):
            if m == 0:
                Phi[row_idx, L] = u_l[i, 0] * np.diff(tk[i])
            else:
                Phi[row_idx, m + L] = (
                    u_l[i, m]
                    * np.sqrt(T)
                    / (1j * m * W / L)
                    * (e_l(m, tk[i][1:]) - e_l(m, tk[i][:-1]))
                )

        Phi[row_idx, L + 1 :] = np.conj(Phi[row_idx, (L - 1) :: -1])

    Phi = np.conj(Phi)

    if RandomThreshold:

```

```

    # this is the small Tikhonov regularization for stability
    PhiPhi = np.conj(Phi.T) @ Phi
    h_l = np.linalg.lstsq(
        PhiPhi + len(q) * SmoothFactor * np.eye(len(PhiPhi)),
        np.conj(Phi.T) @ q,
        rcond=rcond,
    )[0]
else:
    h_l = np.linalg.pinv(Phi, rcond=rcond) @ q

# and then this is to time domain on t_Ph
h_l_pad = np.zeros(t_Ph.shape, dtype=complex)
h_l_pad[: L + 1] = h_l[L:]
h_l_pad[-L:] = h_l[:L]
h_l_pad = np.conj(h_l_pad)

Ph_rec = np.real(fft.ifft(h_l_pad)) * len(t_Ph) / np.sqrt(T)

# should replicate the circular shift so that time axis lines up with t_Ph
idx_shift = int(np.floor(((t_Ph[0] + T / 2) % T) / dt) + 1)
Ph_rec = np.concatenate([Ph_rec[idx_shift:], Ph_rec[:idx_shift]])

print(f"Number of spikes needed: {2*L+2}")
print(f"Number of spikes used: {N_spk}")

return Ph_rec, N_spk

# here is the dendritic filter + IAF parameters

dt = 1e-5
Delta_f = 2.0      # these are all of our desired line spacing in Hz
N_sig = 4          # and this is the number of different stimuli

tau_h = 0.2
c_h = 1e3

delta = 0.01
bias = 1.0
kappa = 1.0
sigma_delta_rel = 0.1    # here we can see the std of threshold noise is 0.1 * delta

def dendritic_filter(t, tau=tau_h, c=c_h):
    """this is the causal kernel h(t) from the homework statement."""
    h = np.zeros_like(t)
    mask = t >= 0
    h[mask] = c * (t[mask] / tau) * np.exp(1.0 - 60.0 * t[mask] / tau)
    return h

# One full experiment for a given max frequency

def run_experiment(f_max, seed_offset=0):
    """
    this is going to help us run the full pipeline for a given max frequency f_max (Hz).

    Should basically return:
        t, t_filt, h, Ph, Ph_rec, u, v, s
    """
    # here we are going to have to choose L so that f_max / L ≈ Δf (constant line spacing)
    L = int(round(f_max / Delta_f))
    f_max = L * Delta_f
    W = 2 * np.pi * f_max
    T = 2 * np.pi * L / W    # = 1 / Δf

```

```

print(f"\n f_max = {f_max:.1f} Hz,  $\Omega$  = {W:.2f} rad/s, L = {L}, T = {T:.3f} s ")

#and these are our time grids: stimulus grid t, filter grid t_filt
t      = np.arange(-T / 2, T / 2, dt)
t_filt = np.arange(-T / 4, 3 * T / 4, dt)  # same length as t, shifted

#random trig stimuli u^i(t)
u_l = []
u    = []
for n in range(N_sig):
    _u_l, _u = random_trig_signal(L, T, t, rng=4020 + seed_offset + n)
    u_l.append(_u_l)
    u.append(_u)
u_l = np.vstack(u_l)
u    = np.vstack(u)

# theh true dendritic filter h(t) on t_filt
h = dendritic_filter(t_filt)

# the projection P h on the same grid
Ph = project_signal(h, L, T, t_filt)

# and these are our filtered outputs v^i(t) = (u^i * h_causal)(t)
v = np.zeros_like(u)
mask_causal = t_filt >= 0
h_causal = h[mask_causal]
for i in range(N_sig):
    v[i] = cconv(u[i], h_causal, dx=dt)

# tis should help us encode with IAF with noisy thresholds
s = np.zeros_like(u, dtype=bool)
voltage = np.zeros_like(u)
delta_v = [None] * N_sig

for i in range(N_sig):
    s[i], voltage[i], delta_v[i] = ideal_iaf_rt_encode(
        v[i],
        t,
        b=bias,
        d=delta,
        k=kappa,
        DeltaVar=sigma_delta_rel * delta,
        RandomInit=True,
        rng=10 + seed_offset + i,
    )

# and this shoud help us identify P h* on t_filt
smooth = 1e-11 if L <= 12 else 1e-8

Ph_rec, N_spk_used = identify_h_ideal_iaf_trig(
    t,
    t_filt,
    u,
    W,
    L,
    bias,
    delta,
    kappa,
    s,
    SmoothFactor=smooth,
    RandomThreshold=True,

```

```

        rcond=1e-9,
    )

    print(f"Number of spikes used: {N_spk_used}")

    return t, t_filt, h, Ph, Ph_rec, u, v, s

# Here is the run for 25 Hz and 50 Hz bands

t_25, t_filt_25, h_25, Ph_25, Phrec_25, u_25, v_25, s_25 = run_experiment(25.0)
t_50, t_filt_50, h_50, Ph_50, Phrec_50, u_50, v_50, s_50 = run_experiment(50.0)

# and here is a quick sanity check: single plot for 25 Hz case
plt.figure(figsize=(7, 4))
plt.plot(t_filt_25, h_25, "k--", label="true h(t)")
plt.plot(t_filt_25, Ph_25, "b", label="P h (projection)")
plt.plot(t_filt_25, Phrec_25, "r", label="P h* (identified)")
plt.xlim(-0.05, 0.15)
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.legend()
plt.title("Filter identification,  $\Omega \approx 2\pi \cdot 25$  rad/s ( $\Delta f = 2$  Hz)")
plt.tight_layout()
plt.show()

# This should be an example of the IAF trace for one of the 25 Hz stimuli

s_demo, v_demo, delta_seq = ideal_iaf_rt_encode(
    v_25[0],
    t_25,
    b=bias,
    d=delta,
    k=kappa,
    DeltaVar=sigma_delta_rel * delta,
    RandomInit=True,
    rng=12345,
)

t_spk_demo = t_25[s_demo]
delta_spk_demo = np.array(delta_seq[:-1])

fig, axes = plt.subplots(2, 2, figsize=(11, 6))
ax_a, ax_b, ax_c, ax_e = axes.ravel()

# (a) input signals
ax_a.plot(t_25, u_25[0], label="u1(t)")
ax_a.plot(t_25, u_25[1], label="u2(t)")
ax_a.set_xlim(-0.1, 0.1)
ax_a.set_ylabel("Amplitude")
ax_a.set_title("(a) Input signals,  $\Omega \approx 2\pi \cdot 25$  rad/s")
ax_a.legend(loc="upper right")

# (b) biased filter output
ax_b.plot(t_25, v_25[0] + bias, label="(u1 * h)(t) + b")
ax_b.axhline(bias, color="r", linestyle="--", label="Bias b")
ax_b.axhline(0.0, color="k", linestyle=":")
ax_b.set_xlim(-0.1, 0.1)
ax_b.set_ylabel("Amplitude")
ax_b.set_title("(b) Biased filter output v1(t) + b")
ax_b.legend(loc="upper right")

# (c) IAF membrane voltage + thresholds

```

```

ax_c.plot(t_25, v_demo, label="membrane v(t)")
ax_c.hlines(delta, t_25[0], t_25[-1], colors="r", linestyle="--", label="mean  $\delta$ ")
ax_c.scatter(t_spk_demo, delta_spk_demo, color="r", s=15, zorder=3, label=" $\delta_k$  at spikes")
ax_c.set_xlim(-0.1, 0.1)
ax_c.set_ylabel("Amplitude")
ax_c.set_title("(c) Ideal IAF neuron response to  $u^1$ ")
ax_c.legend(loc="upper right")

# (d) the filter comparison
ax_e.plot(t_filt_25, h_25, "k--", label="true h(t)")
ax_e.plot(t_filt_25, Ph_25, "b", label="P h (projection)")
ax_e.plot(t_filt_25, Phrec_25, "r", label="P h* (identified)")
ax_e.set_xlim(-0.05, 0.15)
ax_e.set_xlabel("Time [s]")
ax_e.set_ylabel("Amplitude")
ax_e.set_title("(d) Original filter vs. identified filter")
ax_e.legend(loc="upper right")

plt.tight_layout()
plt.show()

# 1) and this again si teh filter comparison for both bandwidths (stacked)

fig, axes = plt.subplots(2, 1, figsize=(7, 6), sharex=True)
ax1, ax2 = axes

ax1.plot(t_filt_25, h_25, "k--", label="true h(t)")
ax1.plot(t_filt_25, Ph_25, "b", label="P h (projection)")
ax1.plot(t_filt_25, Phrec_25, "r", label="P h* (identified)")
ax1.set_xlim(-0.05, 0.15)
ax1.set_ylabel("Amplitude")
ax1.set_title("Filter identification,  $\Omega \approx 2\pi \cdot 25$  rad/s ( $\Delta f = 2$  Hz)")
ax1.legend(loc="upper right")

ax2.plot(t_filt_50, h_50, "k--", label="true h(t)")
ax2.plot(t_filt_50, Ph_50, "b", label="P h (projection)")
ax2.plot(t_filt_50, Phrec_50, "r", label="P h* (identified)")
ax2.set_xlim(-0.05, 0.15)
ax2.set_xlabel("Time [s]")
ax2.set_ylabel("Amplitude")
ax2.set_title("Filter identification,  $\Omega \approx 2\pi \cdot 50$  rad/s ( $\Delta f = 2$  Hz)")
ax2.legend(loc="upper right")

plt.tight_layout()
plt.show()

# 2) here is the absolute error vs time in dB

def plot_abs_error_db(t_filt, h, Ph, Phrec, title, ax, ylim=(-130, 5)):
    """Plot |h-Ph|, |h-Ph*|, |Ph-Ph*| in dB (normalized by max|h|)."""
    eps = 1e-15
    max_h = np.max(np.abs(h))

    e_h_Ph = np.abs(h - Ph) / max_h
    e_h_Phrec = np.abs(h - Phrec) / max_h
    e_Ph_Phrec = np.abs(Ph - Phrec) / max_h

    ax.plot(t_filt, 20 * np.log10(e_h_Ph + eps), label="|h - P h|")
    ax.plot(t_filt, 20 * np.log10(e_h_Phrec + eps), label="|h - P h*|")
    ax.plot(t_filt, 20 * np.log10(e_Ph_Phrec + eps), label="|P h - P h*|")

    ax.set_ylabel("Error [dB]")

```

```

    ax.set_title(title)
    ax.set_ylim(*ylim)
    ax.grid(True)
    ax.legend(loc="lower right")

fig, axes = plt.subplots(2, 1, figsize=(8, 6), sharex=True)

plot_abs_error_db(
    t_filt_25, h_25, Ph_25, Phrec_25,
    "Normalized absolute error (dB),  $\Omega \approx 2\pi \cdot 25$  rad/s",
    axes[0]
)

plot_abs_error_db(
    t_filt_50, h_50, Ph_50, Phrec_50,
    "Normalized absolute error (dB),  $\Omega \approx 2\pi \cdot 50$  rad/s",
    axes[1]
)

axes[1].set_xlabel("Time [s]")
plt.tight_layout()
plt.show()

# 3) This si goign to show us the MSE vs stimulus bandwidth  $\Omega$ 

# and then were we are going to pick a few bandwidths between 10 and 80 Hz
f_list = [10.0, 15.0, 20.0, 25.0, 35.0, 50.0, 70.0, 80.0]

omegas = []
mse_Phrec_h = [] # MSE(P h*, h)
mse_Phrec_Ph = [] # MSE(P h*, P h)

for f_max in f_list:
    t_tmp, t_filt_tmp, h_tmp, Ph_tmp, Phrec_tmp, u_tmp, v_tmp, s_tmp = run_experiment(f_max)

    max_h_tmp = np.max(np.abs(h_tmp))

    e1 = (h_tmp - Phrec_tmp) / max_h_tmp
    e2 = (Ph_tmp - Phrec_tmp) / max_h_tmp

    mse_Phrec_h.append(np.mean(e1**2))
    mse_Phrec_Ph.append(np.mean(e2**2))

    omegas.append(2 * np.pi * f_max)

omegas = np.array(omegas)
mse_Phrec_h = np.array(mse_Phrec_h)
mse_Phrec_Ph = np.array(mse_Phrec_Ph)

eps = 1e-20
mse_Phrec_h_db = 10 * np.log10(mse_Phrec_h + eps)
mse_Phrec_Ph_db = 10 * np.log10(mse_Phrec_Ph + eps)

plt.figure(figsize=(7, 4))
plt.plot(omegas, mse_Phrec_h_db, "o-", label="MSE(P h*, h)")
plt.plot(omegas, mse_Phrec_Ph_db, "s-", label="MSE(P h*, P h)")
plt.xlabel("Stimulus bandwidth  $\Omega$  [rad/s]")
plt.ylabel("Normalized MSE [dB]")
plt.title("Normalized MSE vs stimulus bandwidth  $\Omega$ ")
plt.grid(True)
plt.legend()

```

```
plt.tight_layout()
plt.show()
```

$f_{\max} = 24.0$ Hz, $\Omega = 150.80$ rad/s, $L = 12$, $T = 0.500$ s

Number of spikes needed: 26

Number of spikes used: 199

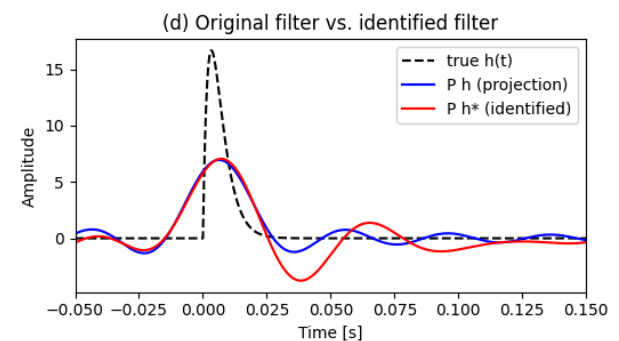
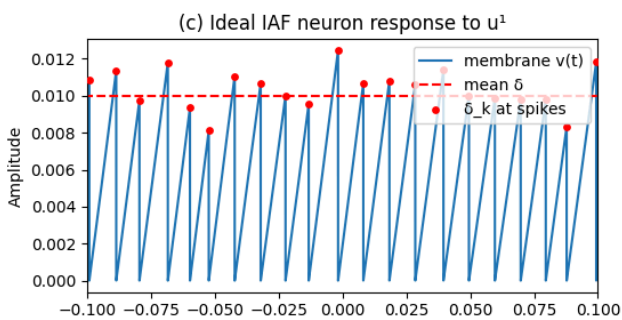
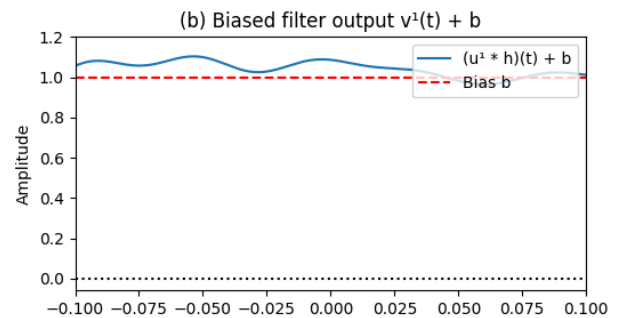
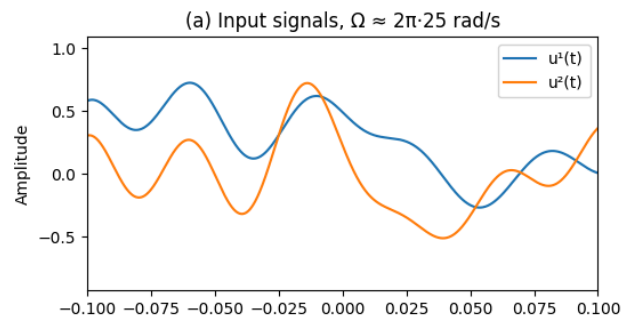
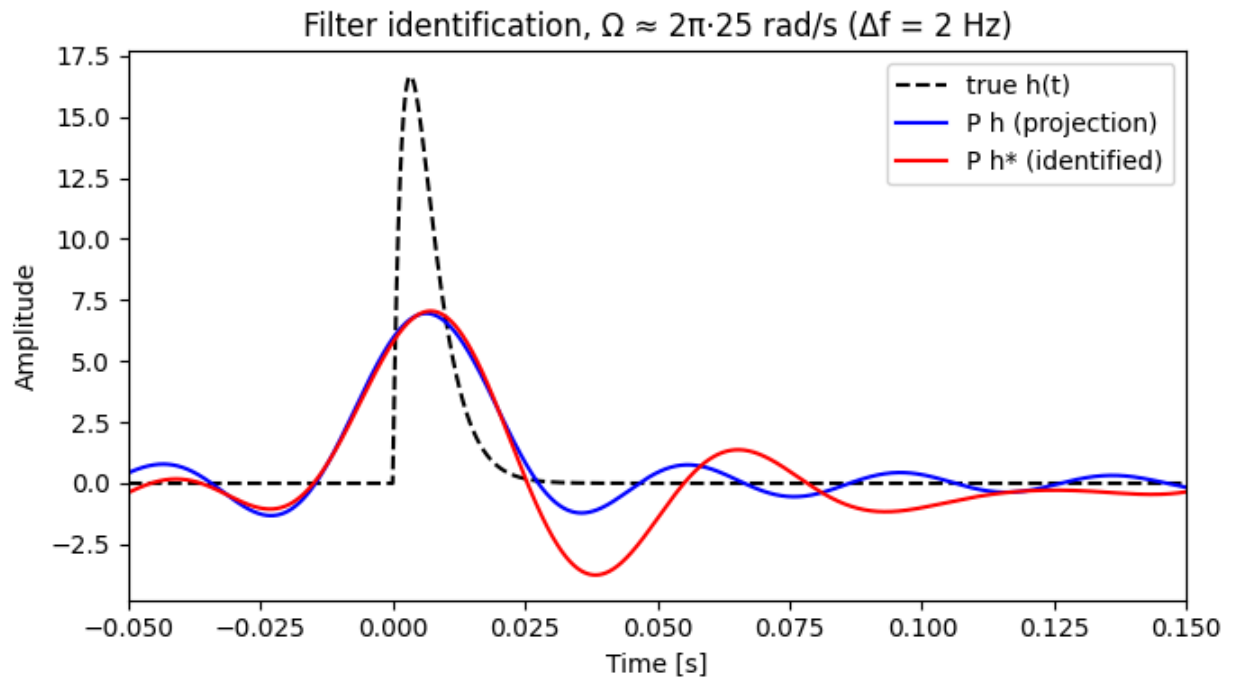
Number of spikes used: 199

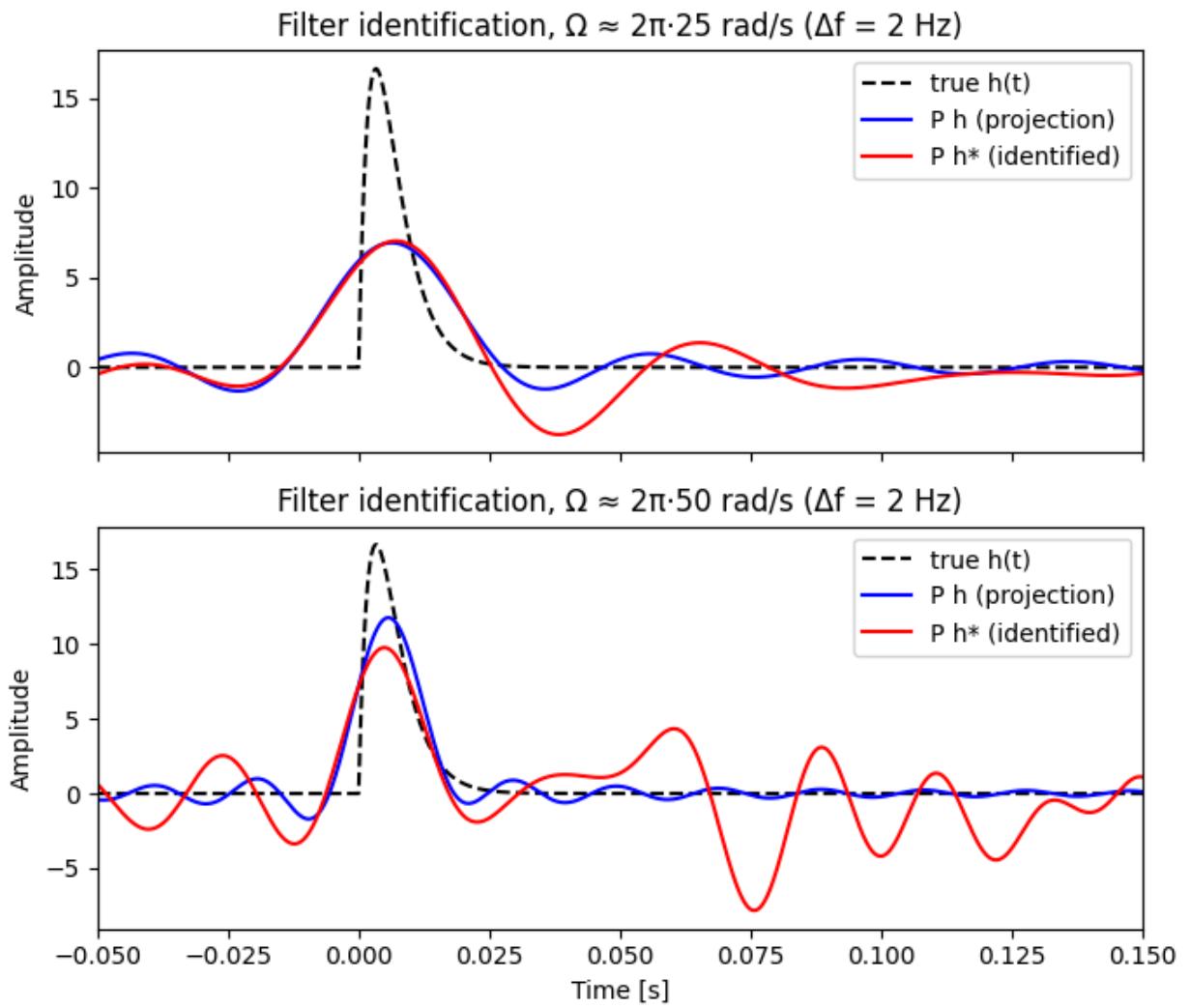
$f_{\max} = 50.0$ Hz, $\Omega = 314.16$ rad/s, $L = 25$, $T = 0.500$ s

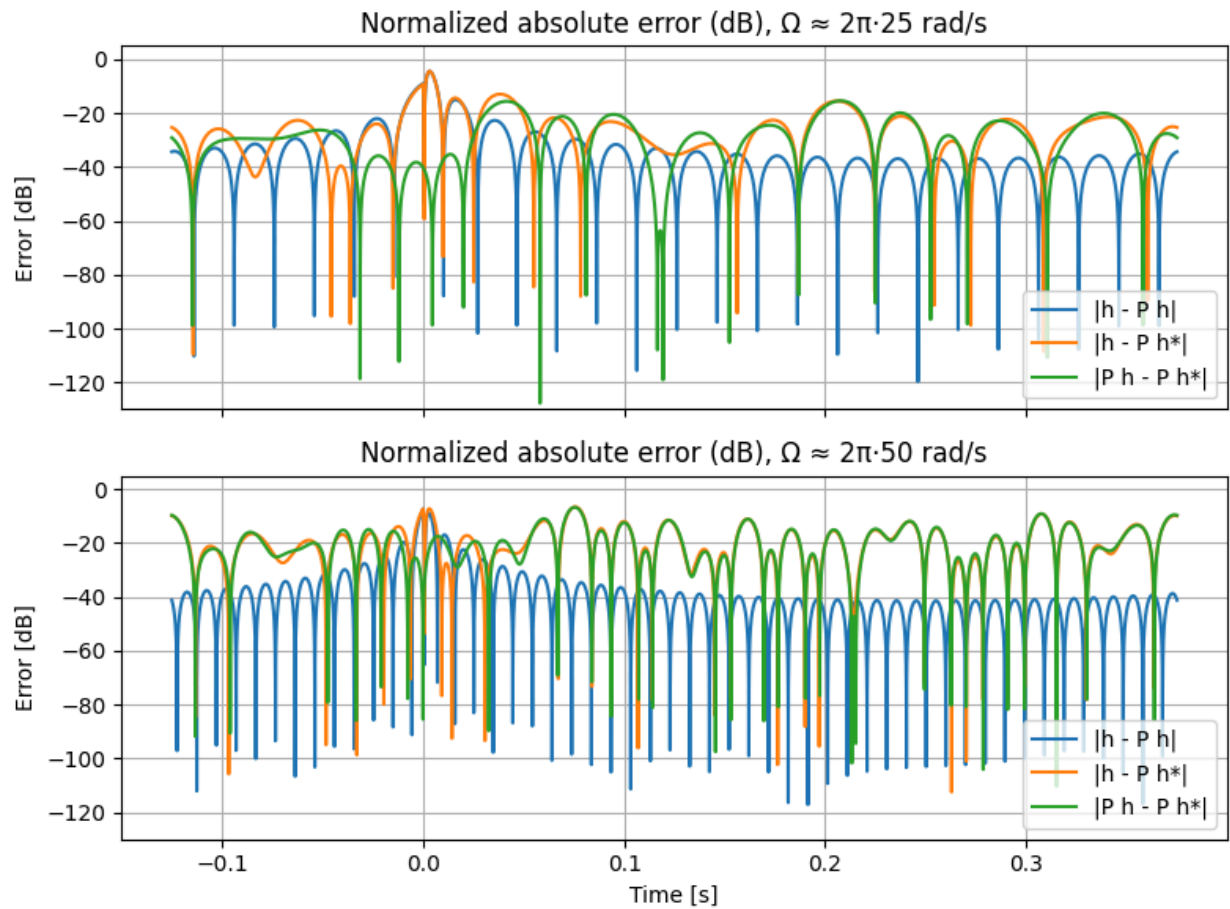
Number of spikes needed: 52

Number of spikes used: 199

Number of spikes used: 199







$f_{\max} = 10.0$ Hz, $\Omega = 62.83$ rad/s, $L = 5$, $T = 0.500$ s
 Number of spikes needed: 12
 Number of spikes used: 200
 Number of spikes used: 200

$f_{\max} = 16.0$ Hz, $\Omega = 100.53$ rad/s, $L = 8$, $T = 0.500$ s
 Number of spikes needed: 18
 Number of spikes used: 200
 Number of spikes used: 200

$f_{\max} = 20.0$ Hz, $\Omega = 125.66$ rad/s, $L = 10$, $T = 0.500$ s
 Number of spikes needed: 22
 Number of spikes used: 200
 Number of spikes used: 200

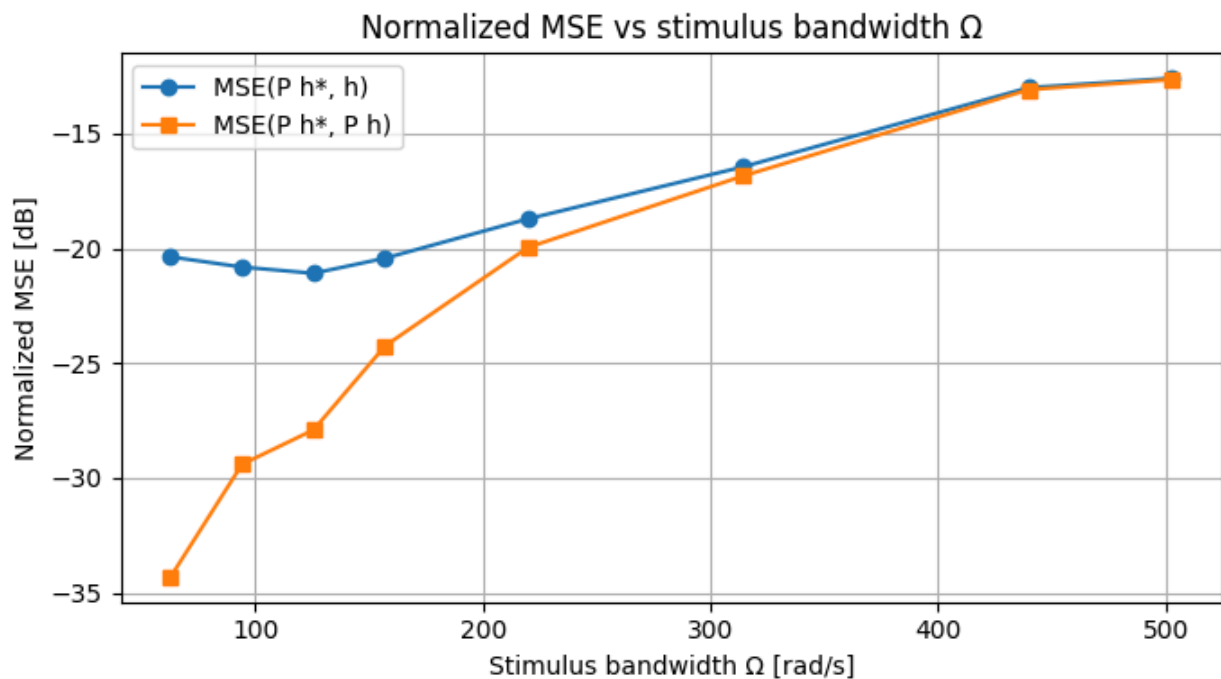
$f_{\max} = 24.0$ Hz, $\Omega = 150.80$ rad/s, $L = 12$, $T = 0.500$ s
 Number of spikes needed: 26
 Number of spikes used: 199
 Number of spikes used: 199

$f_{\max} = 36.0$ Hz, $\Omega = 226.19$ rad/s, $L = 18$, $T = 0.500$ s
 Number of spikes needed: 38
 Number of spikes used: 198
 Number of spikes used: 198

$f_{\max} = 50.0$ Hz, $\Omega = 314.16$ rad/s, $L = 25$, $T = 0.500$ s
 Number of spikes needed: 52
 Number of spikes used: 199
 Number of spikes used: 199

$f_{\max} = 70.0$ Hz, $\Omega = 439.82$ rad/s, $L = 35$, $T = 0.500$ s
 Number of spikes needed: 72
 Number of spikes used: 199
 Number of spikes used: 199

$f_{\max} = 80.0$ Hz, $\Omega = 502.65$ rad/s, $L = 40$, $T = 0.500$ s
 Number of spikes needed: 82
 Number of spikes used: 199
 Number of spikes used: 199



We can assume a causal dendritic filter

$$h(t) = \mathbf{1}_{t \geq 0} c \frac{t}{\tau} e^{1-60t/\tau}, \quad \tau = 0.2, \quad c = 10^3,$$

and we also can assume an ideal IAF neuron with noisy thresholds

$$\delta_k \sim \mathcal{N}(\delta, (0.1\delta)^2), \quad \delta = 0.01,$$

bias $b = 1$, and gain $\kappa = 1$. The input stimuli $u^i(t)$ are trigonometric polynomials of order L .

1. Identification of $\mathcal{P}h^*$ for $\Omega = 2\pi \cdot 25$ and $\Omega = 2\pi \cdot 50$ rad/s

For each of our target bandwidth $f_{\max}$ (25 Hz and 50 Hz) we are going to choose

$$L \approx \frac{f_{\max}}{\Delta f}, \quad \Delta f = 2 \text{ Hz},$$

so that the consecutive spectrum lines end up more spaced by a constant gap Δf and then the effective period is going to be

$$T = \frac{2\pi L}{\Omega} = \frac{1}{\Delta f} = 0.5 \text{ s}.$$

For each of the bandwidths we:

1. Generate $N_{\text{sig}} = 4$ random trigonometric-polynomial inputs $u^i(t) \in \mathcal{T}_L$.
2. Then we filter them with h to obtain $v^i(t) = (u^i * h)(t)$.
3. Pass v^i through the IAF with noisy thresholds in order to obtain spike times $\{t_k^{(i)}\}$.
4. We can then use the Chapter-15 identification algorithm in order to estimate the bandlimited filter $\mathcal{P}h^*(t)$ from the spike data.

The stacked “filter identification” plots compare $h(t)$ (black dashed), and its projection $\mathcal{P}h(t)$ (blue), and the estimate $\mathcal{P}h^*(t)$ (red):

- For $\Omega = 2\pi \cdot 25$ rad/s, $\mathcal{P}h^*(t)$ matches $\mathcal{P}h(t)$ well: since the main peak and side lobes line up, so that means that the spikes contain enough information to recover h within this bandwidth.
- For $\Omega = 2\pi \cdot 50$ rad/s, $\mathcal{P}h^*(t)$ shows more oscillations. Here we see that L is larger, so more Fourier coefficients must be estimated from roughly the same noisy spike budget, which makes the inversion less stable.

2. Finding the absolute error in dB

In order to quantify time-domain accuracy we compute normalized absolute errors

$$e_{h,\mathcal{P}h}(t) = \frac{|h(t) - \mathcal{P}h(t)|}{\max_t |h(t)|}, \quad e_{h,\mathcal{P}h^*}(t) = \frac{|h(t) - \mathcal{P}h^*(t)|}{\max_t |h(t)|},$$

$$e_{\mathcal{P}h,\mathcal{P}h^*}(t) = \frac{|\mathcal{P}h(t) - \mathcal{P}h^*(t)|}{\max_t |h(t)|},$$

and plot $20 \log_{10} e(t)$ for both of the bandwidths.

For $\Omega = 2\pi \cdot 25$ rad/s the curves are mostly between about -20 dB and -40 dB, and $|\mathcal{P}h - \mathcal{P}h^*|$ is often below $|h - \mathcal{P}h|$. This means the dominant error is the bandlimited approximation $h \mapsto \mathcal{P}h$, not the identification from spikes.

For $\Omega = 2\pi \cdot 50$ rad/s the error $|\mathcal{P}h - \mathcal{P}h^*|$ increases: once we push the bandwidth higher, estimation error from noisy, finite spikes that up becoming more important than the projection error itself.

3. MSE vs the stimulus bandwidth Ω

We then repeat the experiment for the effective stimulus frequencies

$f_{\max} \in \{10, 16, 20, 24, 36, 50, 70, 80\}$ Hz, with $\Omega = 2\pi f_{\max}$ in each case, again making sure to keep the line spacing $\Delta f = 2$ Hz. For each of the bandwidth we compute normalized mean-squared errors

$$\text{MSE}(\mathcal{P}h^*, h) = \mathbb{E} \left[\left(\frac{h(t) - \mathcal{P}h^*(t)}{\max_t |h(t)|} \right)^2 \right],$$

$$\text{MSE}(\mathcal{P}h^*, \mathcal{P}h) = \mathbb{E} \left[\left(\frac{\mathcal{P}h(t) - \mathcal{P}h^*(t)}{\max_t |h(t)|} \right)^2 \right],$$

and then plot both curves in dB versus Ω .

From the “Normalized MSE vs stimulus bandwidth” figure:

- At low bandwidths (10–20 Hz), we see that $\text{MSE}(\mathcal{P}h^*, \mathcal{P}h)$ is very small (around -30 dB); and the main error comes from h vs. $\mathcal{P}h$.
- As the Ω increases (around $2\pi \cdot 25$ – $2\pi \cdot 35$ rad/s), $\text{MSE}(\mathcal{P}h^*, h)$ improves because of the trig-polynomial space that we can approximate h better, but $\text{MSE}(\mathcal{P}h^*, \mathcal{P}h)$ grows due to the harder estimation problem that we now have with (larger L with noisy spikes).
- For the largest bandwidths (50–80 Hz) we see that the two curves meet near -15 dB: the model space fits h very well, but the estimation noise and finite spike counts dominate.

All error of our curves are computed **after normalizing by the peak of the true filter**.

In particular, we can see that for any pair of filters $f(t)$ and $g(t)$ we are able to define the pointwise normalized error as

$$(f, g)(t) = \frac{f(t) - g(t)}{\max_t |h(t)|},$$

so that every error is then going to be scaled by $\max_t |h(t)|$.

And then we find that the absolute-error plots in dB use $|h(t) - \mathcal{P}h(t)|$, $|h(t) - \mathcal{P}h^*(t)|$, and $|\mathcal{P}h(t) - \mathcal{P}h^*(t)|$ is then computed in this way.

For the MSE, we can then repeat the identification for several of the stimulus bandwidths

$\Omega = 2\pi f_{\max} \in [2\pi \cdot 10, 2\pi \cdot 80]$ rad/s and then we can plot the normalized mean-squared errors.

$\text{MSE}(\mathcal{P}h^*, h)$ and $\text{MSE}(\mathcal{P}h^*, \mathcal{P}h)$ in dB on the same figure. Both of the curves are going to become less negative as Ω increases, which then means that the reconstruction error is going to grow with bandwidth. Intuitively, when the stimulus excites at the higher frequencies, we find that the IAF neuron has to encode more detailed temporal information with the same noisy spike train, so that recovering the filter becomes more hard and the MSE increases.